

Шифрование гаммированием

Хамза хуссен

16 марта 2026, Москва, Россия

Российский Университет Дружбы Народов

Цель работы

Изучение и практическое применение методов программной реализации алгоритма шифрования гаммированием конечной гаммой.

Задание

1. Реализовать алгоритм шифрования гаммированием конечной гаммой.

Теоретическое введение

Теоретическое введение

Входной текст :A T T A С K A T D A W N

Гамма(ключ): G A M M A G A M M A G A

G T F M C Q A F P A C N

XOR(exclusive)

1 XOR 1 = 0

0 XOR 0 = 0

1 + 0 = 1

xor A->0
G->6

6 -> G

xor T->19
A->0

19 -> T

xor T->19
M->12

31 % 26 -> 5 -> F

xor K->10
G->6

16 -> Q

Рис. 1: алгоритм шифрования гаммированием

Выполнение лабораторной работы

```
ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
N = len(ALPHABET)

def normalize(text: str) -> str:
    text = text.upper()
    return "".join(ch for ch in text if ch in ALPHABET)

def char_to_int(ch: str) -> int:
    return ALPHABET.index(ch)

def int_to_char(x: int) -> str:
    return ALPHABET[x % N]

def gamma_encrypt_finite(plaintext: str, gamma: str) -> str:
    """
    Finite-gamma (phrase) encryption:
    C_i = (P_i + G_i) mod N
    Gamma is repeated to match plaintext length.
    """
    pt = normalize(plaintext)
    g = normalize(gamma)

    out = []
    for i, ch in enumerate(pt):
        p = char_to_int(ch)
        gi = char_to_int(g[i % len(g)])
        c = (p + gi) % N
        out.append(int_to_char(c))
    return "".join(out)
```

Рис. 2: шифрование

```
def gamma_decrypt_finite(ciphertext: str, gamma: str) -> str:
    """
    Finite-gamma decryption:
     $P_i = (C_i - G_i) \bmod N$ 
    """
    ct = normalize(ciphertext)
    g = normalize(gamma)

    if not g:
        raise ValueError("Gamma phrase must contain at least one A-Z letter.")

    out = []
    for i, ch in enumerate(ct):
        c = char_to_int(ch)
        gi = char_to_int(g[i % len(g)])
        p = (c - gi) % N
        out.append(int_to_char(p))
    return "".join(out)

plaintext = "ATTACKATDAWN"
gamma = "GAMMA"

cipher = gamma_encrypt_finite(plaintext, gamma)
back = gamma_decrypt_finite(cipher, gamma)

print("Plaintext:", normalize(plaintext))
print("Gamma    :", normalize(gamma))
print("Cipher   :", cipher)
print("Decrypt  :", back)

Plaintext: ATTACKATDAWN
Gamma    : GAMMA
Cipher   : GTFMCQAF PACN
Decrypt  : ATTACKATDAWN
```

Рис. 3: дешифрование

Выводы

Изученил и разработал методы программной реализации алгоритма шифрования гаммированием конечной гаммой.

Список литературы

https://en.wikipedia.org/wiki/XOR_cipher